

EMCOMM FIELD NODE

An offline emergency communications workstation — capability, deployment and limits

Summary

Emergency communications work begins at the moment the infrastructure everyone else depends on stops answering. The radios keep working. The computer beside the radio usually does not — not because it breaks, but because nearly everything useful on it quietly assumed a connection.

This project turns a clean Linux laptop into a communications workstation that assumes the opposite. Maps, reference material, message handling, position and time all come from the machine itself. It is built once, while a network still exists, in a single pass measured in an hour rather than a day — and it is built the same way every time, so two operators arriving at the same incident have the same node, and a fault understood on one is understood on all of them.

It is radio-agnostic. It does not care which transceiver you own. It cares that there is one, that it has a data or accessory port, and that something can key it.

The problem

A node is needed precisely when the things a laptop leans on are gone. What goes away is broader than "the internet":

- **Map and imagery services.** Every mainstream mapping tool streams tiles. With no route to them, an operator has a blank window and a paper map in the truck.
- **Reference material.** Frequency plans, agency procedures, equipment manuals, ICS forms — all of it normally a search away, none of it reachable.
- **Time.** NTP pools are unreachable. A clock that drifts out of a digital mode's timed transmit window makes a station deaf and inaudible, with no error message to explain it.
- **Software.** Anything not installed before deployment is not going to be installed. There is no package server and no second attempt.

The usual answer is to build a node by hand ahead of time. That works, and it costs a day, and it produces something undocumented. Two nodes built by the same person a week apart differ, and nobody learns which differences matter until one of them is in a parking lot at 0200 and the other is not.

The deeper problem is that a hand-built node cannot be *checked*. There is no list of what was supposed to be true, so there is no way to confirm it still is — after an OS update, after a laptop swap, after the machine sat in a cabinet for eight months.

Who it is for, and when it is used

Written for volunteer and auxiliary communications: ARES, RACES and CERT-style groups, EOC support positions, and any deployment where internet and cellular service cannot be assumed.

Deployment	What the node contributes
Shelter or EOC support position	Offline mapping of the affected area, the reference library, message handling by keyboard mode when voice nets are saturated, and a station log that survives the shift change
Mutual aid, out of area	A node that matches the ones already on scene, with maps staged for an area the operator may never have worked, and no dependence on local infrastructure
Extended field or search operation	Position and time from its own receiver, mesh peer positions plotted on the map, and a reference library that does not need a signal
Training, drill and exercise	The same build under realistic conditions, so the exercise tests procedure rather than someone's laptop
Readiness and bench work	A single pass rebuilds a node, or re-runs one step of it, which makes a cabinet full of laptops maintainable rather than archaeological

It suits a group that wants **standardization without a full-time IT volunteer**. The build is reproducible, the result is inspectable, and a node handed to a new operator behaves like the one they trained on.

What a provisioned node can do

The stack is assembled from established open-source tools; the contribution here is that they arrive configured, verified, and stocked with offline data.

Capability	Provided by
Position and disciplined time with no network	gpsd and chrony bound to a declared GPS receiver
Weak-signal keyboard messaging on HF	JS8Call
Automatic link establishment on HF	ion2G, with a staged channel plan
Packet and APRS over a sound-card interface	Direwolf, an AX.25 software TNC
LoRa mesh text and position	Meshtastic tooling for an external mesh node

Capability	Provided by
Offline topographic and imagery mapping	QMapShack, with local tile pyramids registered for you
Offline encyclopaedic reference	Kiwix ZIM archives
Offline document library	A loopback-only web server over a staged PDF collection
Aircraft situational awareness	ADS-B reception on a software-defined radio
Independent weather imagery	Direct satellite reception and decoding
Transceiver programming	CHIRP
Station record	QLog

Position and time it can defend

A GPS receiver gives the node its own position and, through `chrony`, its own time — reaching stratum one without a network. This is the least glamorous capability and the one most likely to decide whether a digital mode works at all, because the timed modes need the clock to be right and will not tell you when it is not.

Passing traffic when voice will not

Three independent paths, chosen by band and conditions rather than by preference:

- **HF keyboard messaging** for regional and long-haul text when a voice net is unworkable — weak-signal modes get through at signal levels that voice cannot.
- **HF automatic link establishment**, for scheduled or unattended links using a staged channel plan.
- **Packet and APRS on VHF or UHF**, through any sound-card interface, for local traffic, bulletins and position reporting.

All three attach to the transceiver the group already owns. The node supplies the software, the configuration and the channel plan; the operator declares which audio device and which keying line to use.

A mesh layer that does not need a repeater

Tooling for an external LoRa mesh node covers short text and position among a team on foot, independent of the licensed bands and of any infrastructure. Where mapping is also installed, peer positions are written out for import into the map — a file the operator refreshes, not a live overlay, which is stated plainly here because it is the sort of thing that disappoints in the field if it is promised as more.

The map, without the map servers

Offline raster tiles are fetched for a declared operating area before deployment and registered with the mapping application so a layer is drawn on first launch. The view opens on the operating area with a grid over it. Nothing is streamed, nothing is cached from a live service, and nothing changes when the network goes.

The operating area is chosen by the operator, by coordinates and radius, so a node built for one region is not carrying somebody else's terrain.

Answers without a search engine

Two reference systems, because they fail differently:

- **An encyclopaedic archive** in ZIM format, read locally — the general question, asked offline.
- **A document library** of PDFs served on loopback only, for the specific material a group actually needs: frequency plans, agency procedures, and equipment manuals for gear nobody has memorized.

Loopback-only is deliberate. The library is for the operator at the keyboard, not a service offered to whatever network the node later finds itself on.

Receive-only situational awareness

A software-defined receiver supports two uses the node can make of the spectrum without transmitting: **aircraft tracking** via ADS-B, useful where air assets are working an incident, and **direct weather satellite reception**, which produces imagery of local conditions from the spacecraft rather than from a forecast service that is no longer reachable.

Both drive the same receiver and only one can hold it at a time, so aircraft tracking is left off at boot and started deliberately. A node that grabbed the receiver at startup would cost a satellite pass silently.

The station record

Logging is provisioned alongside the operating software, so a deployment produces a record rather than a stack of notes — which matters most when the shift changes and the next operator needs to know what has already been passed.

How a node is deployed

Three phases, and the order is the point.

1. **Build while a network still exists.** Run the provisioner, select the steps this node needs, and let it install, configure and stage. Map tiles, reference archives and the document library are pulled during this phase and are the bulk of the time and disk.

2. **Verify before it leaves the bench.** A read-only pass checks what actually landed on disk and reports; it never repairs. A written checklist covers the remainder — the parts no automated check can see, including anything requiring a radio.
3. **Operate cold.** From here the node needs no network and asks for none.

Steps are independent and individually selectable, so a node can be repaired, extended or partially rebuilt later without starting again. A group running several nodes can build them to a common baseline and let individual machines differ where the assignment differs.

Every run writes a transcript. If a build goes wrong, that file is what gets sent to whoever is helping.

What it is not

- **Not a radio, and not a substitute for one.** It is the workstation beside the radio.
- **Not a license and not training.** Transmitting on the amateur bands requires an operator license; the software says so where a callsign is entered rather than only in documentation.
- **Not an offline copy of the internet.** It carries what was staged onto it, chosen by the group that built it.
- **Not a turnkey appliance.** It is a reproducible build with an honest account of its own state, which is a different and more useful thing.

Design commitments

Three rules shape the build. Each was bought with a failure.

- **A verification row reads content, not paths.** The recurring defect in software of this kind is a check that passes for work that did not happen: a staged profile that copied cleanly and substituted nothing, a tracking file that existed and held zero entries, a map source registered under a key the installed version does not read. A full run ends with several dozen such rows, and the count grows with every check added. What matters is not the number but that each row reads what is inside the file rather than confirming the file is there.
- **The node is told about its hardware; it never goes looking.** The operator names the GPS receiver and the radio interface. The reason is not tidiness: opening a serial port asserts its control lines, and a sound-card interface keys its transmitter from one of them, so a provisioner that swept the serial devices hunting for a radio could put a station on the air — into an antenna that may not be connected, under a callsign that may be a placeholder. Declared bindings also survive the fact that a GPS puck and a radio interface both enumerate the same way, and which one comes up first depends on plug order.

- **No operator or machine data is committed.** Shipped configuration carries substitution tokens filled in at provisioning time, and verification fails if a token survives. Device bindings that carry a serial number are excluded from version control. This is a rule with history: real data from a working node was once captured into this repository and required a full history rewrite to remove.

What has been proven, and what has not

A project of this kind is only as trustworthy as its account of its own limits. The distinction maintained throughout is between what has been **observed working on hardware** and what has been **written and checked but never exercised**.

Observed on hardware: provisioning completing and reporting honestly; ADS-B tracking live aircraft and releasing its receiver cleanly; weather satellite software built from source with its receiver visible and a full tracking set loaded; a kernel driver blacklist surviving a device replug and a reboot; a GPS receiver reaching stratum one within about a minute of a cold boot with no network, twice; offline maps drawing from local tiles with the network pulled.

That last one is worth stating precisely, because it is the kind of claim this section exists to discipline. When it was first recorded, the map sources were listed and the operator selected the layer by hand. The activation keys being written were ones the mapping application's shipped version does not read — it renamed that configuration group in a later release — so nothing was activated, and a listed source is indistinguishable from an active one at a glance. The layer now draws unselected, and that has been observed on a node. The original record was true about the tiles and wrong about the activation, and only a direct before-and-after on one machine separated the two.

Not observed, and stated as such:

- **Dock-triggered autostart has never fired,** on any hardware. The step installs four artifacts and verification confirms all four exist. Whether a dock insertion launches anything is unknown, and no dock is available to the project to test against. It ships selectable and documented as in development, because a convenience feature failing to start is not a node that cannot communicate.
- **No radio has been keyed.** Audio and keying bindings are declared, substituted and path-checked. Nothing in this project has put a signal on the air.
- **Verification checks the filesystem, not radio frequency.** No automated check covers anything requiring a transmitter or an antenna.

Where a claim is inherited rather than measured — a figure from a datasheet, a hardware identifier from a walkthrough rather than a dump of the real device — it is labeled as such in the source that uses it.

Limits

- **One reference platform.** The stack targets Linux Mint 22.x (XFCE) on x86_64, typically a rugged ex-fleet laptop. Other hardware is untested rather than unsupported, and one optional step is written for a specific dock.
- **One test bed.** Findings come from a small number of machines operated by one person. Failures that only appear at scale have not been seen.
- **Steps are not dependency-checked.** Selecting a later step without an earlier one may produce a node that verifies cleanly and does not work. That is a deliberate trade for the ability to re-run a single step.
- **Offline data is operator-supplied.** Map tiles, reference archives and application profiles are fetched or staged by the operator. The repository ships the mechanism, not the content.

Licensing

Released under the **GNU General Public License v3.0 or later**. The license text ships with every release archive; it is deliberately not excluded from the packaged artifact.

Two standalone helper scripts — the map tile fetcher and the QMapShack map diagnostic — are MIT-licensed so they can be reused outside this project; the MIT text ships beside the GPL. An organization whose policy does not permit GPL-3.0 software can request separate terms through the project's issue tracker.

Some capabilities require an amateur radio license to operate legally on the air. Receiving is unrestricted; transmitting is not.

Closing

The claim this project makes is narrow and worth stating plainly: a field node should be reproducible, and a tool that builds one should be honest about what it has and has not established. A group that deploys these knows what is on every machine, can rebuild one in an hour, and has a written account of which capabilities have been seen to work and which are still promises.